

Cairo University
Faculty of Engineering
Computer Engineering Department

Data Structures and Algorithms

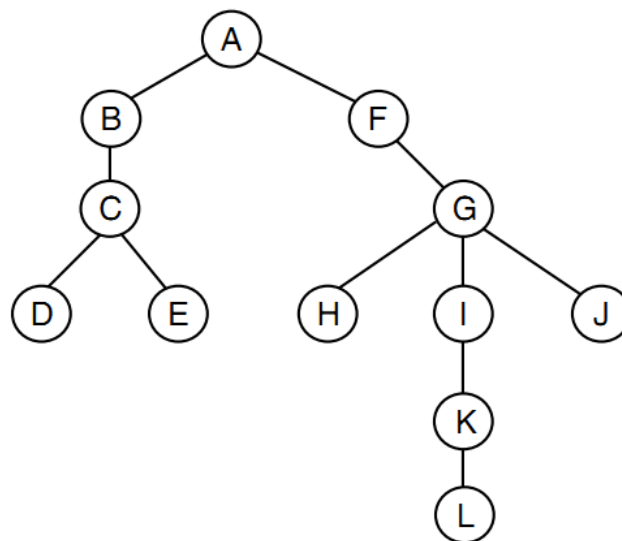
Sheet #5- Trees

Part I: Exercises

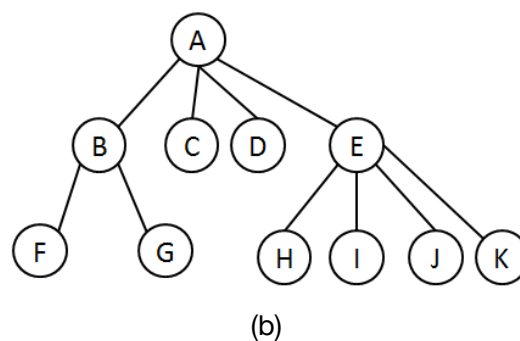
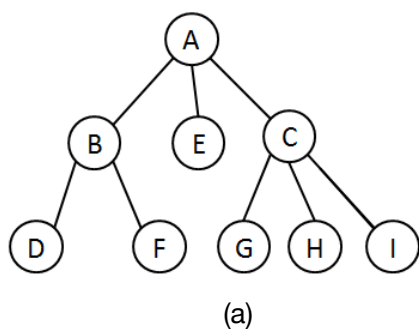
A. General Trees

1. For the following **tree**, get the following:

- [1] Tree nodes
- [2] Tree Branches
- [3] Root
- [4] Leaves
- [5] Internal nodes
- [6] Parents
- [7] Children
- [8] **Parent** of G
- [9] **Children** of G
- [10] **Ancestors** of H
- [11] **Descendants** of F
- [12] **Siblings** of G
- [13] **Siblings** of I
- [14] **Sub-trees** of node A
- [15] **Sub-trees** of node G
- [16] **Sub-trees** of node H
- [17] **In-degree** of G
- [18] **In-degree** of A
- [19] **Out-degree** of G
- [20] **Out-degree** of H
- [21] **Degree** of G
- [22] **Level** of node A
- [23] **Level** of node K
- [24] **The height** of the tree
- [25] **The height** of the sub-tree G (rooted at G)
- [26] **The indented list representation**

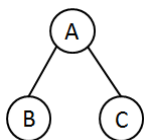


2. Convert the following trees into binary trees.

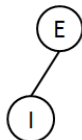


B. Binary Trees

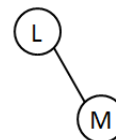
1. For a **binary tree** of N nodes (from 1 to N):
 - a. What is the max height? Draw its tree.
 - b. What is the min height? Draw its tree.
2. For a **binary tree** of height H :
 - a. What is the max number of nodes? Draw its tree.
 - b. What is the min number of nodes? Draw its tree.
3. For each of the following **binary trees**:
 - a. Is it **full** or **complete**?
 - b. What is its **balance factor**?
 - c. Is it a **balanced binary tree**?



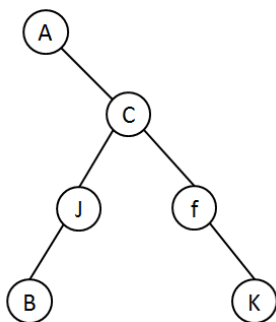
(a)



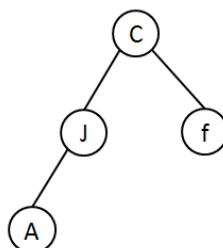
(b)



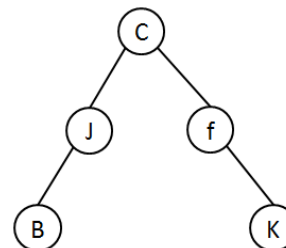
(c)



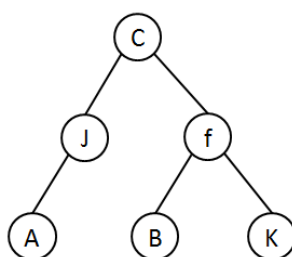
(d)



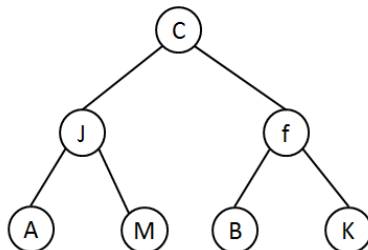
(e)



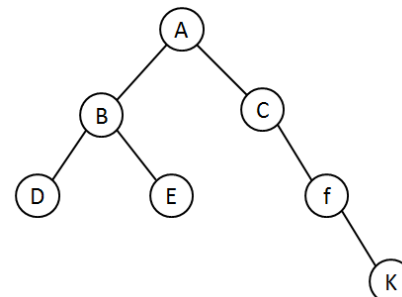
(f)



(g)



(h)



(i)

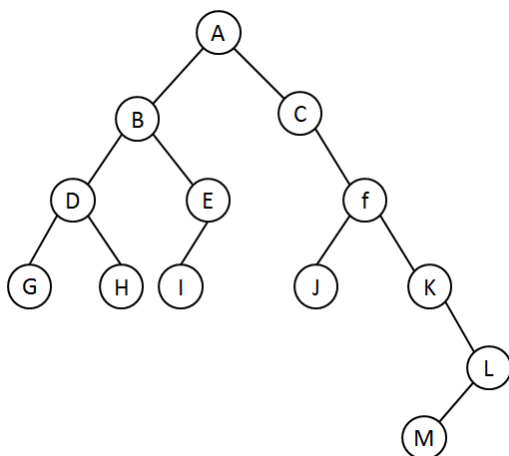
4. Draw **all** “**complete**” trees of height 3.
5. Draw “**full**” trees of the following heights: 2, 3 and 4.

6. For the following binary trees, write:

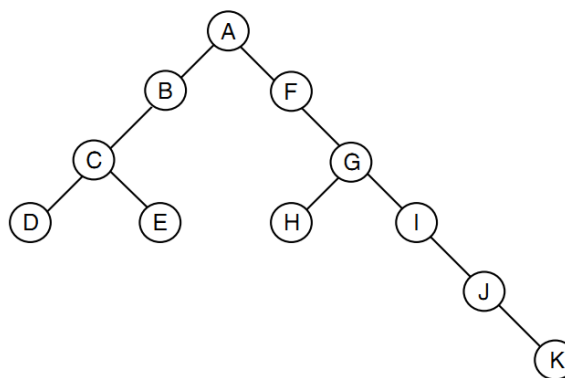
a. The **depth-first** traversals:

- i. Pre-order
- ii. In-order
- iii. Post-order

b. The **breadth-first** traversal



(a)



(b)

7. Find the root of each of the following **binary** trees:

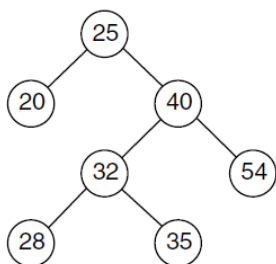
- a. tree with **post-order** traversal: FCBDG
- b. tree with **pre-order** traversal: IBCDFEN

C. Binary Search Trees (BST)

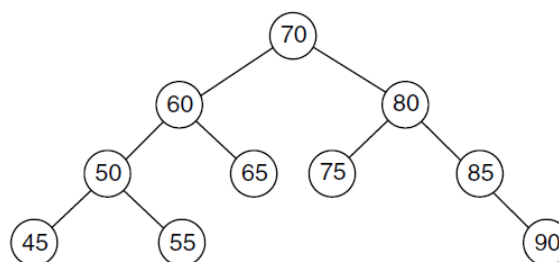
1. Create a **BST** using the following data entered as a sequential set: 14 23 7 10 33 56 80 66 70
2. Draw the BST that results when you insert items with keys: E A S Y Q U E S T I O N
3. Traverse the **BST** no. (a) in the figure below using an **in-order** traversal.

4. For the **BST** no. (b) in the figure below:

- a. Search for the value 56 (show the compared nodes in order)
- b. Get the smallest value
- c. Get the largest value
- d. Insert 44, 66, and 77 into the binary search tree
- e. Delete the node containing 60 from the binary search tree
- f. Delete the node containing 85 from the binary search tree



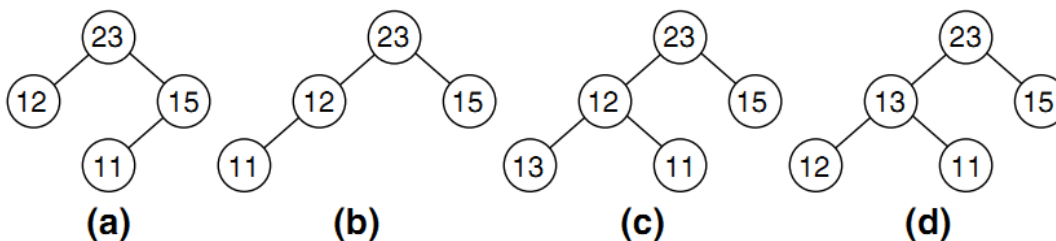
(a)



(b)

D. Heap

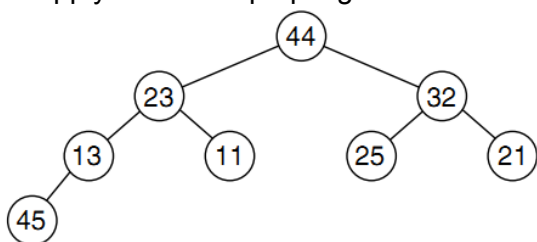
1. Define AVL trees and Heap trees. Draw an example for each.
2. Show which of the following figures are heaps and which are not.



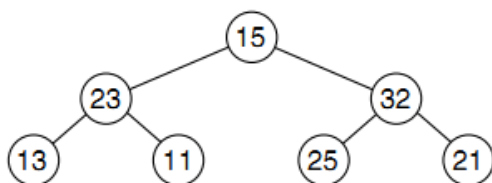
3. Make a heap out of the following data read from the keyboard:

23 7 92 6 12 14 40 44 20 21

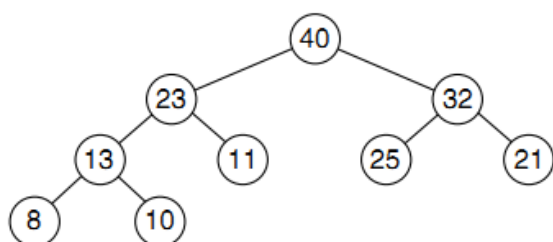
4. Apply the re-heap up algorithm to the non-heap structure shown in following figure.



5. Apply the re-heap down algorithm to the partial heap structure shown in the following figure.



6. For the following figure:
 - a. Write the array representation of the heap
 - b. Apply the delete operation to the heap. Repair the heap after the deletion.
 - c. Insert 38 into the heap. Repair the heap after the insertion.



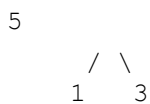
Part II: Coding Problems

- Problems in this part are for self-training to practice coding on Trees
- Not to be solved during the problem set session
- We will just explain why each required member function needs to be written as 2 functions: one private recursive (does all the work) and one public that just calls the private function. Then we will solve problem 1 as an example.
- You may try them during lab sessions
- Answers will be provided to you for guidance

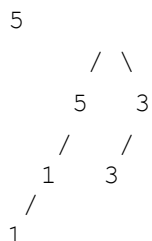
A. Binary Trees

1. Write a member function that counts the number of nodes in a binary tree.
2. Write a member function that returns the sum of all the nodes in a binary tree.
3. Write a member function that counts the number of leaves in a binary tree.
4. Write a member function that counts the number of leaves that contain even values in a binary tree.
5. Write a member function that returns the maximum value of all the nodes in a binary tree. Assume all values are non-negative; return -1 if the tree is empty.
6. Write a member function that prints all the keys that are less than a given value, V , in a binary tree.
7. Write a member function that returns true if the sum of all the nodes in a binary tree equal to a given value, V .
8. Write a member function **is_mirror** that takes a binary tree **T** and checks whether it is a mirror of the calling tree or not, It returns true if the two trees are mirrors; every left child in the first tree is a right child in the second tree and vice versa.
9. Write a member function that creates a mirror image of a binary tree. All left children become right children and vice versa. The input tree must remain the same.
10. Write a member function **double_tree** that, for each node in a binary tree, it creates a new duplicate node and insert it as the left child.

So the tree...



Is changed to...



11. Write a member function to delete all the leaves from a binary tree, leaving the root and intermediate nodes in place. (**Hint:** Use a preorder traversal.)
12. Write a member function **same_structure** that returns true if the two passed tree are structurally the same whatever the data inside them.
13. Write a member function **has_path_sum** that returns true if the tree contains any path from the root to one of the leaves with the given sum.
14. Given a binary tree, write a member function that prints out all of its root-to-leaf paths, one per line.
15. The height of a tree is the maximum number of nodes on a path from the root to a leaf node. Write a member function that returns the height of a binary tree.
16. A **balanced binary tree** is a tree in which all the following conditions are satisfied:
 - a. the left and right sub-trees heights differ by at most one level
 - b. the left sub-tree is balanced
 - c. the right sub-tree is balancedWrite a member function that checks if a given tree is balanced or not.
17. A **full** binary tree is a tree in which every node other than the leaves has two children. Write a member function that checks whether a given tree is full or not.
18. Write a member function that determines whether a binary tree is **complete**.
19. Rewrite the binary tree **pre-order, in-order and post-order** traversal member functions using a **stack** instead of recursion.

B. Binary Search Trees (BST)

1. Write a member function that inserts a specific value in BST tree.
2. Write a member function that searches for a specific value in BST tree.
3. Write a member function that returns the minimum data value found in BST tree.
4. Write a member function that returns number of nodes less than a specific value in BST tree
5. Write a member function that returns a sub-tree that has nodes sum equal to an input value.